



Blazing Fast PSI from Improved OKVS and Subfield VOLE

Srinivasan Raghuraman
Visa Research, USA

Peter Rindal
Visa Research, USA

ABSTRACT

We present new semi-honest and malicious secure PSI protocols that outperform all prior works by several times in both communication and running time. Our semi-honest protocol for $n = 2^{20}$ can be performed in 0.37 seconds compared to the previous best of 2 seconds (Kolesnikov et al., CCS 2016). This can be further reduced to 0.16 seconds with 4 threads. Similarly, our protocol sends $187n$ bits compared to $426n$ bits of the next most communication-efficient protocol (Rindal et al., Eurocrypt 2021). Additionally, we apply our new techniques to the circuit PSI protocol of Rindal et al. and observe a $6\times$ improvement in running time. These performance results are obtained by two types of improvements.

The first is an optimization to the protocol of Rindal et al. to utilize *sub-field* vector oblivious linear evaluation. This optimization allows our construction to be the first to achieve a communication complexity of $O(n\lambda + n \log n)$ where λ is the *statistical* security parameter. In particular, the communication overhead of our protocol does not scale with the computational security parameter times n .

Our second improvement is to the OKVS data structure which our protocol crucially relies on. In particular, our construction improves both the computation and communication efficiency as compared to prior work (Garimella et al., Crypto 2021). These improvements stem from algorithmic changes to the data structure along with new techniques for obtaining both asymptotic and tight concrete bounds on its failure probability.

KEYWORDS

Private Set Intersection; OKVS; VOLE

ACM Reference Format:

Srinivasan Raghuraman and Peter Rindal. 2022. Blazing Fast PSI from Improved OKVS and Subfield VOLE. In *Proceedings of Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3548606.3560658>

1 INTRODUCTION

Private-Set Intersection. In this work, we present new improvements for efficiently performing private set intersection (PSI). Here, two mutually distrusting parties, each hold a set of values X, Y respectively and wish to learn the intersection between their sets $X \cap Y$ without revealing any additional information. In particular, the first party (receiver) with set X should not learn anything about

$X \setminus Y$ beyond the size of Y . Similarly, the second party (sender) should only learn the size of X .

PSI dates back to the 1980s [18] and was initially based on OPRFs/Diffie–Hellman. In fact, several modern protocols [3, 6, 14] are still based on the protocol of [18]. With the invention of oblivious transfer extension [15], research into a new protocol family was initiated by Schneider et al. [25] along with many derivatives [16, 20, 23, 26, 27]. These protocols offer improved efficiency at the expense of increased communication.

However, with the advent of [24] and the closely related protocols of [12, 28], the situation has begun to change. [24] introduced a new data structure called an OKVS which offers a very convenient way to represent the sets X and Y . [24] went on to combine their OKVS and the PSI protocol of [23] to obtain one of the most efficient PSI protocols of their time. Shortly after, [28] was able to observe that the OKVS data structure can be more efficiently combined with vector oblivious linear evaluation (VOLE) [2, 5] to obtain an even more efficient PSI protocol. Concurrently, [12] proposed an improvement to OKVS which allowed for an improved communication overhead of the [24] PSI protocol. Despite all these advances, the non-OKVS based protocol of [16] remains the most computationally efficient while [28] is the most communication efficient, and [12] is a compromise between the two.

We make the observation that recent improvements to VOLE [5] and OKVS [12] can be applied to [28]. While this does further reduce the communication complexity of [28], the running time of the protocol remains slower than [16] due to the computational overhead of the [12] OKVS data structure. We will present significant improvements to the OKVS of [12, 24] along with new techniques for further reducing the communication overhead of [28].

We further note that our improvements also translate to several other PSI related functionalities. One of the most important is circuit PSI [13, 23, 28]. The functionality is similar to normal PSI except the output is secret shared between the parties. This can be particularly useful when additional encrypted computation needs to be performed on the intersection before revealing it, e.g. compute the cardinality or the sum of associated values [14]. We apply our new techniques to this protocol and observe a $6.8\times$ and $2.3\times$ reduction in running time and communication for set size of a million.

Other functionalities include the multi-party PSI protocol of [19] makes use of both OKVS and a primitive known as an OPRF/OPPRF which can utilize our improvements. We leave the application of our work to this protocol and many others as future work.

Oblivious Key-Value Stores. The aforementioned data structure known as an Oblivious Key-Value Stores (OKVS) consists of algorithms Encode and Decode. Encode takes a list of key-value pairs $(k, v) \in L$ as input and returns an abstract data structure P which encodes L . Decode takes such a data structure and a key k' as input, and gives some output v' . Decode can be called on any key, but if it is called on some k' that was used to generate P , then the result is the corresponding v' . An OKVS scheme is said to be *linear* if for

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '22, November 7–11, 2022, Los Angeles, CA, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9450-5/22/11...\$15.00

<https://doi.org/10.1145/3548606.3560658>

any k , $\text{Decode}(P, k)$ can be expressed as a linear combination of P , i.e. $\text{Decode}(P, k_i) = \langle P, \text{row}(k_i) \rangle = v_i$ where row is some public function. In this work we restrict ourselves to linear OKVS schemes.

In PaXoS [24], n items are encoded into a vector $\vec{P}$ of length $m = 2.4n$. This scheme is parameterized by $\text{row} : \{0, 1\}^* \rightarrow R$ with $R \subset \{0, 1\}^m$ containing only weight 2 vectors. Decoding of a key k is therefore the sum $P_i + P_j = \langle \vec{P}, \text{row}(k) \rangle$ where $\text{row}(k)$ is 1 at positions i, j . Encoding is performed using a graph traversal of the *cuckoo graph* implied by the n keys and row .

However, with high probability this process will fail to encode a small subset of the input. The size of this subset is known to be at most $O(\log n)$. PaXoS solves this issue by changing the distribution of row such that it outputs a uniformly random weight 2 vector of length $2.4n$ followed by $O(\log n) + \lambda$ randomly sampled bits, i.e. $m = 2.4n + O(\log n) + \lambda$. Intuitively, this allows the encoder solve to the system of equations corresponding to these last $O(\log n)$ vertices using the last $O(\log n) + \lambda$ random bits while all the other vertices are solved using the graph algorithm above.

[12] proposed generalizing the above construction to have the main part of $\text{row}(k)$ output weight 3 vectors as opposed to weight 2. [12] proposes that m can be reduced to $m = 1.3n + O(\log n) + \lambda$ which is a reduction of almost $2\times$. However, they were unable to prove the security of their construction for practical security levels. Instead they empirically show that their construction fails with small but noticeable probability and then provably amplify the failure probability to be negligible using a recursive amplification technique. Unfortunately, this amplification further increases the running time and compactness m .

1.1 Our Contributions

In this work, we propose an improved OKVS construction and use it, along with subfield-VOLE, as a building block in order to obtain the fastest and most communication efficient PSI protocols to date.

- We put forth a framework to both theoretically and concretely understand failure probabilities associated with the randomized OKVS construction based on cuckoo hashing.
- In the range of parameters/distribution we work in, our OKVS encoder is “optimal” with overwhelming probability.
- Equipped with the understanding from above, we derive OKVS constructions that considerably outperform the state of the art in terms of compactness and running time.
- Our OKVS includes several optimization over prior art, including reducing the total row weight by $\sim 10\times$, $GF(128)$ dense columns, no recursive structure, and clustering.
- We obtain the most efficient PSI protocol to date by employing our improved OKVS in addition to a new optimization we introduce to the OPRF/PSI construction put forth in [28]. These improvements enable our construction to perform a PSI of $n = 2^{20}$ items in 0.37 and 0.16 seconds in the single and multi threaded settings, respectively. On the same hardware, the previous fastest protocol required 2 seconds, a $5\times$ improvement, and requires more communication.
- We further optimize the protocol for the low communication setting using subfield-VOLE. We achieve the lowest communication complexity to date, performing a PSI of n items with $O(n\lambda + n \log n)$ bits of communication, where λ is the

statistical security parameter. This protocol remains highly efficient in terms of running time. A PSI with $n = 2^{20}$ can be performed with just 180n bits of communications compared to the previous best [28] of 426n.

- Finally, we apply our new OKVS construction to the circuit PSI protocol of [23, 28] and observe a $1.5\times$ and $23\times$ improvement for running time and communication.

1.2 Overview

In Section 2 we detail our new OKVS scheme. $\text{Encode}(\{(k_1, v_1), \dots, (k_n, v_n)\})$ takes as input a set of key-value pairs. The algorithm constructs a matrix H based on the values of the keys $k_1, \dots, k_n$, where the i th row $H_i = \text{row}(k_i)$ for some function row . The output is a vector $\vec{P}$ such that $H\vec{P} = (v_1, \dots, v_n)$, i.e. $\langle \text{row}(k_i), \vec{P} \rangle = v_i$. This is achieved by *efficiently* placing H into *lower triangular (aka reduced row-echelon) form*. Crucially, our linear time algorithm for triangulating H only permutes the rows and columns of H plus a small amount of additional work, and therefore H remains sparse. Once in this form, $\vec{P}$ can be computed using the standard *back substitution* algorithm. We also present several optimizations to the distribution of H which makes this process much more efficient.

In Section 3 we prove that our technique for placing H in lower triangular form is asymptotically optimal. We then translate these asymptotically results into concrete and tight parameters for our constructions using a combination of empirical and analytical methods. Unlike some prior works, our construction directly scales to any security level and works for a wide range of parameters which can be tuned in an application specific manner.

In Section 4 we present how to apply our OKVS scheme to the PSI protocol of [28]. From a technical perspective, this application is relatively straight forward. However, it immediately gives a large running time and communication improvement over prior art. This construction achieves both semi-honest and malicious security.

We then present an optimization that further reduces the communication overhead of our PSI protocol in the semi-honest setting. Intuitively, the PSI protocol works by first having the receiver construct an OKVS $\vec{P} \in \mathbb{F}^m$ such that $\text{Decode}(\vec{P}, x) = x$ for each x in their set X . The parties generate a VOLE correlation where the PSI sender holds $\Delta \in \mathbb{F}$, $\vec{B} \in \mathbb{F}^m$ and the receiver holds $\vec{C} \in \mathbb{F}^m$ such that $\vec{C} - \vec{B} = \Delta \vec{P}$ and therefore, since the OKVS is linear, we have

$$\text{Decode}(\vec{C}, x) - \text{Decode}(\vec{B}, x) = \Delta x$$

for $x \in X$. The sender can encode their elements $y \in Y$ as $F(y) = \text{Decode}(\vec{B}, y) - \Delta y$ which will equal $F(x) = \text{Decode}(\vec{C}, x)$ for $y = x \in X$. The PSI protocol completes by having the sender send $Y' = \{F(y) \mid y \in Y\}$ to the receiver who intersects it with $X' = \{F(x) \mid x \in X\}$ and thereby infers the intersection $X \cap Y$.

The original protocol requires all of this computation to be over a field $\mathbb{F}$ of size $O(2^\kappa)$. In addition, to construct the VOLE correlation for the *chosen* vector $\vec{P}$ requires $|\vec{P}|$ communication. We present optimizations that allow the VOLE correlation to remain over the field $\mathbb{F}$ (for security) while reducing the size of $\vec{P}$ to be $(\lambda + \log n)m = O(n\lambda + n \log n)$. In particular, we make use of subfield-VOLE.

1.3 Notation

We use κ as the computational security parameter and λ for statistical security. $[a, b]$ denotes the set $a, a+1, \dots, b$ and $[b]$ is shorthand for $[1, b]$. We denote row vectors $\vec{A} = (a_1, \dots, a_n)$ using the arrow notation while the elements are indexed without it. A set $S = \{s_1, \dots, s_n\}$ will use similar notation. For a matrix M , we use $\vec{M}_i$ to denote its i th row vector, and $M_{i,j}$ for the element at row i and column j . $\langle \vec{A}, \vec{B} \rangle$ denotes the inner product of $\vec{A}, \vec{B}$. We use $=$ to denote the statement that the values are equal. Assignment is denoted as $:=$ and for some set S , the notation $s \leftarrow S$ means that s is assigned a uniformly random element from S . If a function F is deterministic then we write $y := F(x)$ while if F is randomized we use $y \leftarrow F(x)$ to denote $y := F(x; r)$ for $r \leftarrow \{0, 1\}^*$.

2 OUR OKVS CONSTRUCTION

We begin by describing our core OKVS construction which maps a set of key-value pairs $\{(k_1, v_1), \dots, (k_n, v_n)\}$ to a vector $\vec{P} \in \mathbb{F}^m$. The full description of the construction can be found in Figure 1 and definitions in Section A.1.

Our algorithm begins by sampling an instance key $r \leftarrow \{0, 1\}^\kappa$. Let us define $\text{row}(k, r) := \text{row}'(k, r) \parallel \text{row}(k, r) \in \mathbb{F}^m$ where $\text{row}'(k, r) \in \{0, 1\}^{m'}$ outputs a uniformly random (sparse) weight w vector. In practice we will set $m' \approx 1.23n$. $\text{row}(k, r) \in \mathbb{F}^{\hat{m}}$ outputs a short dense vector. The exact distribution of row will vary and is discussed below. $\hat{m}$ can be thought of as a small constant. Next, a matrix H is defined as

$$H := \begin{bmatrix} \text{row}(k_1, r) \\ \dots \\ \text{row}(k_n, r) \end{bmatrix} \in \mathbb{F}^{n \times m}.$$

The algorithm will output $\vec{P} \in \mathbb{F}^m$ such that $H\vec{P} = (v_1, \dots, v_n)$.

The first step is to perform *triangulation*. This permutes the rows and columns of H by π^r, π^c respectively. In particular, it defines two permutation matrices $\pi^r \in \{0, 1\}^{n \times n}, \pi^c \in \{0, 1\}^{m \times m}$. Let T correspond to applying the permutations to H .

$$T := \pi^r \cdot H \cdot \pi^c = \begin{bmatrix} A & B & C \\ D & E & F \end{bmatrix}.$$

Ideally, T will be lower triangular, i.e. reduced row-echelon form. However, we will show that most of the time this is impossible to achieve using only permutations¹. Instead, these permutations will have the structure that T can be decomposed into $A, \dots, F$ such that $F \in \{0, 1\}^{\delta \times \delta}$ is a large lower triangular matrix where $\delta = n - g \approx n$. $g \ll n$ is referred to as the *gap* and is the number of rows that could not be triangulated. The idea is that we can then perform back-substitution on F to encode $\delta \approx n$ of the inputs. The remaining g inputs are encoded using a different mechanism in $O(g^2)$ time.

In more detail, $F \in \{0, 1\}^{\delta \times \delta}$ is lower triangular where $\delta := n - g \approx n$. $B \in \mathbb{F}^{g \times \hat{m}}, E$ will consist of the dense columns generated by row . Our *triangulation* algorithm will aim to minimize the g subject to F being lower triangular. The next phase computes

$$T' := \begin{bmatrix} I & -CF^{-1} \\ 0 & I \end{bmatrix} \cdot T = \begin{bmatrix} A' & B' & 0 \\ D & E & F \end{bmatrix}$$

¹One can always triangulate H using Gaussian elimination, but this would increase the density of H and make back-substitution inefficient.

where $A' := -CF^{-1}D + A, B' := -CF^{-1}E + B$. For some Q , let $B^* := QB'$ be the (lower) reduced row echelon form of B' . If B^* does not have full row rank, e.g. $g > \hat{m}$, the algorithm aborts. Otherwise, let

$$T^* := \begin{bmatrix} Q & 0 \\ 0 & I \end{bmatrix} \cdot T' = \begin{bmatrix} A^* & B^* & 0 \\ D & E & F \end{bmatrix}$$

where both B^*, F are lower triangular with ones on their diagonal. We can apply the same row permutation to $V := (v_1, \dots, v_n)$:

$$V^* := \begin{bmatrix} Q & -QCF^{-1} \\ 0 & I \end{bmatrix} \cdot \pi^r \cdot V$$

Since all of T^* is lower triangular, we can compute

$$T^* \cdot P^* = V^* \quad \Rightarrow \quad P^* = T^{*-1} \cdot V^*$$

in linear time using *back-substitution* by solving one row at a time in a top down manner. Finally, the algorithm output $P := P^* \cdot (\pi^c)^{-1}$.

The running time of this algorithm is $O(g^2 + m)$ plus the time required to compute the π^r, π^c permutations. Therefore, the efficiency of this algorithm depends on the triangularization phase minimizing g . We note that in step 5 of Figure 1 refers to an DOKVS. This optional step is required for our Circuit PSI protocol but not the PSI protocols. See Section A.2 for details.

Triangulation. Our triangulation algorithm runs in time $O(m)$. We prove that this algorithm is optimal for the case of $g = 0$ and [near] optimal with overwhelming probability for the small values of g which we will be concerned about.

The algorithm takes as input the matrix H and outputs a row & column permutation π^r, π^c . These permutations are chosen such that $\pi^r \cdot H \cdot \pi^c$ is in g -approximate lower triangular form, where all but g rows are triangular. Begin by initializing $H' \in \{0, 1\}^{n \times m'}$ as the sparse part of H , i.e. the first m' columns. The algorithm will iteratively remove rows and columns from H' and insert them into $[A \ C \parallel D \ F]$ where A, C, D, F are initially size zero.

The matrix F will have the invariant that it is lower triangular and at iteration i , $F \in \{0, 1\}^{i \times i}$. For a typical iteration, there will exist some column of H' that has hamming weight one. Let the one in this column be located at $H'_{r,c}$, i.e. row r and column c of H' . The idea is that we will remove (set to zero) this row from H' and add a permuted version of it to the matrix $[A \ C \parallel D \ F]$ such that $H'_{r,c}$ is mapped to $F_{1,1}$. In this way the size of F grows by one. It is not hard to verify that F remains lower triangular.

However, in some situations no weight one columns will exist in H' . In this case we will select the lowest non-zero weight column of H' . One of these non-zero rows will be added to F as before while all other non-zero rows are permuted such that they are added to C in an arbitrary order. Triangulation is complete when all rows have been removed from H' . π^r, π^c are defined as the permutations implicit in the transformation above, with the addition that the dense columns of H are mapped to be in between $A \parallel D$ and $C \parallel F$.

In Section 3.3 we prove that this algorithm is optimal for $g = 0$ and performs well for small g . Section 3.2 shows that the general case for $g = O(n)$ is likely intractable to solve optimally. For our construction this turns out to not be an issue due to g being small with overwhelming probability, e.g. $g < 4$, as shown in Section 3.

We note that in the special case that we do have an upper bound $\hat{g}$ on g , it is possible to optimally triangulate H . As we shall see later,

Parameters: Statistical and computational security parameters λ, κ respectively. Input length n with elements $(z_i, v_i) \in \mathcal{Z} \times \mathcal{V}$, a finite field $\mathbb{F}$, a finite group $\mathbb{G}$. For some $m' = O(n)$, $\hat{m} = O(\lambda)$, let the output length be $m = m' + \hat{m}$.

Encode $((z_1, v_1), \dots, (z_n, v_n); r) :$

- (1) **[Sample]** Let $\text{row}' : \mathcal{Z} \times \{0, 1\}^\kappa \rightarrow \mathcal{S}_w$ and $\text{row} : \mathcal{Z} \times \{0, 1\}^\kappa \rightarrow \mathbb{F}^{\hat{m}}$ be random functions where $\mathcal{S}_w \subset \{0, 1\}^{m'}$ is the set of all weight w strings. Let $\text{row}(z, r) := \text{row}'(z, r) \parallel \text{row}(z, r)$ and define

$$H := \begin{bmatrix} \text{row}(z_1, r) \\ \dots \\ \text{row}(z_n, r) \end{bmatrix} \in \mathbb{F}^{n \times m}$$

- (2) **[Triangulate]** Let $H' := H, J := \emptyset$. While H' has rows:
- (a) Select $j \in [m]$ such that the j th (sparse) column of H' has the minimum non-zero weight.
 - (b) Append index j to the ordered list J . Remove all rows $i \in [n]$ from H' for which $H'_{i,j} \neq 0$.
- Define $\delta := |J|$, the gap as $g := n - \delta$, permutation matrices $\pi^r \in \{0, 1\}^{n \times n}, \pi^c \in \{0, 1\}^{m \times m}$ such that $\pi^c_{m-k, m-\delta-k} = 1$ for $k \in [0, \hat{m}]$, $\pi^c_{j_i, m+1-i} = 1$ and $\pi^r_{n+1-i, i'} = 1$ for some i' where $H'_{i', j_i} \neq 0$ and all $i \in [\delta]$. Let

$$T := \pi^r \cdot H \cdot \pi^c = \begin{bmatrix} A & B & C \\ D & E & F \end{bmatrix}.$$

where $F \in \{0, 1\}^{\delta \times \delta}$ is lower triangular, $B \in \mathbb{F}^{g \times \hat{m}}, E \in \mathbb{F}^{\delta \times \hat{m}}$ are the dense columns.

- (3) **[Zero-C]** Compute $T' := \begin{bmatrix} I & -CF^{-1} \\ 0 & I \end{bmatrix} \cdot T = \begin{bmatrix} A' & B' & 0 \\ D & E & F \end{bmatrix}$
- (4) **[Solve-Dense]** If B' doesn't have full row rank, return $\perp$. Let $B^* := QB'$ be the (lower) reduced row echelon form of B' , and
- $$T^* := \begin{bmatrix} Q & 0 \\ 0 & I \end{bmatrix} \cdot T' = \begin{bmatrix} A^* & B^* & 0 \\ D & E & F \end{bmatrix}, v^* := \begin{bmatrix} Q & -QCF^{-1} \\ 0 & I \end{bmatrix} \cdot \pi^r \cdot v$$
- (5) **[Optional: DOKVS]** If a doubly oblivious key value store is required, set $T^* := [I \parallel T^*] \in \mathbb{F}^{m \times m}$ and $v^* := r' \parallel v^* \in \mathbb{F}^m$ where $r' \leftarrow \mathbb{F}^{m-n}$, I is the identity matrix with dimension $(n - m) \times m$.
- (6) **[Back-substitution]** Compute $\tilde{P}^* := T^{*-1} \cdot v^*$ via back-substitution and return $\tilde{P} := \tilde{P}^* \pi^{c-1}$.

Decode $(\tilde{P}, z, r) : \text{Return } (\text{row}(z, r), \tilde{P})$.

Figure 1: OKVS scheme.

one need only try all possible sets of at most $\hat{g}$ rows of the so-called 2-core of G_H (G_H is a hypergraph defined by H), something we will define later in Section 3. This modified triangulation algorithm is described in Figure 5. Let h_2 denote the size of the 2-core of G_H . Then, the complexity of the modified triangulation step would be $O(m + h_2^{\hat{g}})$. As we will describe in Section 3, in the parameter regime we consider, $\hat{g} = O(1)$ and $h_2 \leq c \cdot \hat{g}$ with overwhelming probability for $c = O(1)$. In fact, empirically, $c \approx 2$ with overwhelming probability and $\hat{g}$ is a small constant (for instance, between 2 and 5). The proofs of all of the above facts and the optimality of the modified triangulation can be found in Section 3.

Full row rank of B' . A key requirement of our algorithm is that B' contains an invertible submatrix of size g . A trivial requirement for this is that m' , the number of columns contained in B' , is greater than or equal to g . This is ensured by upper bounding g as $\hat{g}$ and setting m' accordingly. However, it is still possible that B' does not have full row rank even if $m' \geq \hat{g}$. The key factors that determine this are the distribution of row and how large $m' - g$ is. We

present two techniques for ensuring that B' has full row rank with overwhelming probability.

Binary row. The first technique was directly inspired by [24]. Given the upper bound $\hat{g}$ on the gap size g , row is defined as a uniform function $\text{row} : \{0, 1\}^* \rightarrow \mathbb{F}^{m'}$ where $\mathbb{F} := \{0, 1\}$, $m' = \hat{g} + \lambda$ and λ is the statistical security parameter. Given the distribution of row , we can bound the probability of B' having full row rank.

LEMMA 2.1. *The $\Pr[B' \text{ having full row rank}] \geq 1 - 2^{-\lambda}$.*

PROOF. $B' = -CF^{-1}E + B$ is uniformly random since B is. A random $m' \times \hat{g}$ binary matrix has full row rank with probability

$$\frac{\prod_{i=0}^{\hat{g}-1} (2^{m'} - 2^i)}{2^{m'\hat{g}}} \geq 1 - 2^{\hat{g}-m'} = 1 - 2^{-\lambda}$$

Here, the i th term of the numerator is the probability that the i th row is linearly independent of the previous rows while the denominator is the total number of such matrices.

Field row. While the previous approach works, it has the downside of significantly increasing the weight of the row function. This imposes a non-trivial performance overhead due the need for the additional (binary) multiplications. As an alternative, we propose defining $\mathbb{F}$ as a large field, e.g. with order 2^κ , and defining $\text{row}(k, r) = (\hat{k}, \hat{k}^2, \dots, \hat{k}^{m'})$ where $m' := \hat{g}$ is an upper bound on the size of the gap, $\hat{k} := I(k, r)$, and $I : \{0, 1\}^* \rightarrow \mathbb{F}$ is a random mapping. In this case, $B' = -CF^{-1}E + B$ will have full row rank with overwhelming probability. This is because B is a random Vandermonde matrix (which has full row rank iff the $\hat{k}$ values are distinct) and the argument to show that B has full row rank with overwhelming probability readily extends to show that the sum of a fixed matrix (here, $-CF^{-1}E$) and B will also have full row rank with overwhelming probability.

LEMMA 2.2. *The sum of a random Vandermonde matrix and any fixed matrix has full row rank with overwhelming probability.*

PROOF. Recall that we can argue that a random Vandermonde matrix will have full row rank as follows. Consider the sub-matrix formed by the first n columns of the matrix (where n is the number of rows). We will show that this matrix has full column rank, which completes the proof. Suppose there is some linear combination of the columns that yields the zero vector. This then translates to the fact that the Vandermonde polynomial of degree $n - 1$ (or n) has n distinct (with overwhelming probability) roots (other than 0), which is possible iff the polynomial is the zero polynomial. This argument can be extended to show that the sum of a fixed matrix and a random Vandermonde matrix will also have full row rank with overwhelming probability. As before, a column combination can be expanded to a polynomial of degree n which admits at most n distinct roots in $\mathbb{F}$. Since the Vandermonde matrix is randomly sampled, with overwhelming probability, the value sampled will be a non-root of the polynomial under consideration².

The main downside of this approach is the need for performing multiplication over a large field. However, due to the hardware support for $\mathbb{F}_{2^{128}}$ multiplication, the performance of this approach significantly outperforms the binary case.

²Note that we assume r is sampled after the k values are fixed.

2.1 Clustering

Although the aforementioned construction achieves linear time encoding for the desired parameter regime, it has a major limitation when encoding/decoding very large input lengths, e.g. $n = 2^{20}$. The bulk of the memory accesses are effectively random access into an array of size $O(n)$. When n is sufficiently large, the CPU cache can no longer hold the required data and it must be fetched from main memory. This can impose a large slowdown in the running time.

To mitigate the effect of this, we propose restricting the distribution of $\text{row}' : \{0, 1\}^* \rightarrow S_w$ such that the non-zero positions are clustered together. In particular, we redefine $S_w \subset \{0, 1\}^{m'}$ to be the set of all w weight strings such that for all $s \in S_w$, $\text{support}(s) \subset (im^*, im^* + m^*]$ for some $i \in [0, \beta)$, m^* where β is the number of clusters and m^* is the cluster size. That is, each string $s \in S_w$ has all of its non-zero positions clustered in the i th length m^* non-overlapping substring, for some i . Each cluster can then be triangulated independently, where only one cluster needs to be processed at a time. In our implementation we set $m^* \approx 2^{14}$ which gives a good tradeoff between parameters.

While clustering has advantages, it can impact the failure probability. To address this, we modify the distribution of row . The core issue is that each cluster will result in some gap with a probability based on m^* . Given an upper bound n^* on the number of inputs items mapped to any cluster, one can then determine an upper bound on the maximum gap size $\hat{g}$ for any cluster. We consider the following two strategies.

Combined. row can be defined as a uniform function $\text{row} : \mathcal{Z} \rightarrow \mathbb{F}_2^{\hat{m}^*}$ where $\hat{m}^* := \lambda + \hat{g}\beta$. In this formulation we leverage the fact that the λ extra columns can be shared among the β clusters. Since λ is likely several factors larger than $\hat{g}$, this will significantly reduce the weight of the rows. Overall this approach allows for a relatively compact encoding while still enabling clustering.

Separate. Alternatively, row can be defined to have β clusters which are correlated with row' . If $\text{row}'(z)$ is mapped to the i th cluster, then $\text{row}(z)$ should output a uniform random string in $\mathbb{F}_2^{\hat{m}\beta}$ subject to all but the i th non-overlapping substring of length $\hat{m}$ being zero. In this way we are effectively creating β independent instances of the underlying OKVS. This approach is slightly less compact compared to the previous but allows for greater independence between the clusters and a lower row weight.

Overall, clustering allows the algorithm to process chunks of size m^* which can be orders of magnitude smaller than m . This improves the memory locality and cache efficiency. Moreover, say for example we ensure that $m^* = 2^{16}$. It is then the case that all internal state values can be stored in a 16-bit integer, as compared to say 32 or 64-bit, which allows the implementation to have even better memory locality, e.g. by a factor of 2 or 4.

Clustering naturally lends itself to a multi-threaded implementation. Items can be mapped to clusters and then a thread can process each cluster without the need for expensive synchronization. All of these optimizations hold true for both encoding and decoding.

3 ANALYSIS AND PARAMETER SELECTION

The performance of our OKVS scheme critically depends on the upper bound $\hat{g}$ of g . We demonstrate that our algorithm is optimal for several important cases and otherwise achieves an extremely good

approximation in practice. We then give a detailed characterization of the distribution of $\hat{g}$.

3.1 The Case of $w = 2$

We begin with the case of row weight $w = 2$. This case is qualitatively distinct due to the expected value of g being non-zero for practical values of e and the optimality of our algorithm.

Reformulating as a Graph Problem. In order to analyze the gap g , we reformulate our problem as a graph problem as follows. Recall that the sparse binary part of the matrix H under consideration is of size $n \times m'$ where $m' = en$ for an expansion factor of $e > 1$. We sample the sparse part of H as follows: Each row is sampled independently at random subject to the constraint that the row is of weight $w = 2$. Now, consider the graph $G_H = (V, E)$ induced by the sparse part of H where $V = [m']$ and

$$E = \{e_i = (e_{i,1}, e_{i,2}) : H_{i,1} = H_{i,2} = 1\}_{i \in [n]}$$

It turns out that the problem of determining how to best solve the system of equations determined by H using the process of back-substitution, that is, finding free variables in H , is equivalent to the problem of optimally *peeling* G_H [29]. We recall that peeling G_H corresponds to iteratively selecting an edge with a vertex of degree 1 (or a *free* vertex) and removing that edge. If this process terminates with all the edges of the graph being removed, the graph is said to have been completely peeled. Clearly, in our setting, this means that there is a way to solve the system of linear equations involving H entirely using back-substitution, or in other words, $g = 0$. Otherwise, the peeling algorithm terminates with a sub-graph of G_H where every vertex has degree at least 2. That sub-graph is called the *2-core* of G_H .

Indeed, different sub-graphs of G_H would admit different peelings that would lead to different such final unpeelable sub-graphs. One can then ask the question of whether one can determine the largest sub-graph of G_H that can be completely peeled. We call this the problem of optimally peeling G_H . Optimally peeling G_H would lead to the identification of a sub-graph (where every vertex has degree at least 2) of minimum size which if removed from G_H would make it completely peelable. It is easy to see that the gap g is the size of this subset, i.e., the minimum number of edges to be removed from this 2-core in order to make it peelable. Immediately, g is upper-bounded by the size of the 2-core of G_H . Thus, in general, one way of upper-bounding g is to get a handle on the size of the 2-core of G_H . But for $w = 2$, we can do a lot better. It turns out that for the case of $w = 2$, it is indeed possible to efficiently compute the optimal peeling of G_H a greedy algorithm. We go on to describe why this is the case.

Matroids and the Greedy Algorithm. We begin by defining the matroid abstraction and cast our algorithm with $w = 2$ as the associated greedy algorithm. This immediately implies that our algorithm is optimal for the case of $w = 2$. A *matroid* $M = (E, I)$ consists of a *ground set* E and a collection $I \subseteq 2^E$ of *independent sets* where each $A \in I$ is a subset of E , i.e., $A \subseteq E$, which satisfies the following three properties:

- (1) The empty set is independent, i.e. $\emptyset \in I$.

- (2) **(hereditary)** Every subset of an independent set is an independent set, i.e. $A' \subseteq A \in I \implies A' \in I$.
- (3) **(augmentation/exchange)** If $A, B \in I$ and $|A| > |B|$, then $\exists a \in A \setminus B$ s.t. $B \cup \{a\} \in I$.

This definition can be extended to the setting where the elements of E each have a positive weight. The weight of any subset $A \subseteq E$ is then defined as the sum of the element weights. Given this, a maximum weight independent set $A \in I$ can be found in $|E|$ time given oracle access to determining set independence [21], i.e., determining if $A \cup \{a\} \in I$ for some $A \in I, a \in E$. In particular, A can be found by initializing $A := \emptyset$ and greedily adding some $a \in E$ to A so long as $A \cup \{a\} \in I$.

Let us now look back to the problem of optimally peeling G_H , or equivalently, finding the largest sub-graph of G_H that is completely peelable. It is a well-known fact that if $G = (V, E)$ is an undirected graph, and F is the family of sets of edges that form forests in G , then (E, F) forms a matroid. For the sake of completeness, we recall the proof of this fact here.

LEMMA 3.1. *If $G = (V, E)$ is an undirected graph, and F is the family of sets of edges that form forests in G , then (E, F) is a matroid. This matroid is called the graphic matroid of G , denoted as $M(G)$.*

PROOF. Clearly, $\emptyset \in F$. F clearly satisfies the hereditary property as removing edges from a forest leaves another forest. F also satisfies the exchange property: if A and B are both forests, and A has more edges than B , then it has fewer connected components, so by the pigeonhole principle there is a component C of A that contains vertices from two or more components of B . Along any path in C from a vertex in one component of B to a vertex of another component, there must be an edge with endpoints in two components, and this edge may be added to B to produce a forest with more edges. Thus, F forms the independent sets of a matroid.

Let us now consider $M(G_H)$. The independent sets of this matroid are the forests in G_H . Now, note that forests are *completely peelable*, i.e., have empty 2-cores. Thus, finding the largest possible forest embedded in G_H corresponds to finding the optimal way to peel G_H . Taking it full circle, this then also gives us the exact set of rows of H which attains the minimum possible gap g . To this end, we can set the weight of each edge $e_i \in E$ as 1. The greedy algorithm will then determine the maximum weight, i.e., largest size independent set $A \subseteq 2^E$. More concretely, let the ground set $E := [n]$ correspond to the rows of H and we define I such that all $A \in I$ correspond to sets of rows which can be made exactly triangular via row a permutation. From the argument above, clearly (1), (2) and (3) are satisfied. We then define the weight of each $a \in E$ as one. The maximum weight independent set $A \in I$ is then exactly the set of rows which minimizes g . It is then an easy task to verify that our triangulation algorithm is equivalent to the matroid greedy algorithm and therefore is optimal. We state this in the following lemma.

LEMMA 3.2. *For the case of $w = 2$, Step 2 of the procedure in Figure 1 optimally triangulates H , that is, it computes the minimum g over all possible choices of the set J .*

Analysis to bound g . From the discussion above, we know that whatever g is, we can efficiently find it. The remaining analysis

thus is to bound g with overwhelming probability. Again, we do this via reformulating the problem as a graph problem. The process of sampling the sparse part of H can be equivalently considered as sampling a random graph $G_H = (V, E)$ where $V = [m']$ and $|E| = n$, that is, where E is a collection of random independently (with replacement) sampled n edges. Now, as argued above, the gap g of H is upper-bounded by the size of the 2-core of G_H . Thus, in order to upper bound g , it suffices to understand the distribution of the size of the 2-core of such a randomly sampled G_H .

This has, once again, been studied extensively. As noted in [11], the size of the 2-core of G_H follows the size of the so-called *giant component* in G_H . We recall the phenomenon of the emergence of the giant component in a random graph. Asymptotically, the evolution of the random graph undergoes a phase transition at $e = 2$. The case of $e > 2$ is called the *sub-critical* phase and $e < 2$ is called the *super-critical* phase. In the sub-critical phase, the random graph consists with overwhelming probability of tree components and components with exactly one cycle. All components during the sub-critical phase are rather small, of order $O(\log m')$, and there is no significant gap in the order of the first and the second largest component. The situation changes when $e < 2$, i.e., when we enter the super-critical phase and then with overwhelming probability, the random graph consists of a single giant component (of the order comparable to $O(m')$), and some number of simple components, i.e., tree components and components with exactly one cycle. One can also observe a clear gap between the order of the largest component (the *giant*) and the second largest component which is of the order $O(\log m')$. This phenomenon of dramatic change of the typical structure of a random graph is called its phase transition. Erdős and Rényi [8], and many others following [1, 17] established the phenomenon of the *double-jump*, where as we pass through the phase transition $e = 2$, the size of the largest component changes from $O(\log m')$ to $m'^{2/3}$ to $\Omega(m')$, and in fact, for $e = 2$, the size of the largest component is $m'^{2/3}$ with overwhelming probability. The size of the 2-core, and the upper bound $\hat{g}$ for the gap of H , grows gradually (unlike the size of k -cores for $k \geq 3$, as noted in [11]) and eventually follows the same distribution as that of the largest component of G_H for $e < 2$. Close to $e = 2$, the upper bound $\hat{g}$ on the gap turns out to be $O(\log m')$, following the size of the largest component when $e > 2$. We note this in the following lemma.

LEMMA 3.3. *For the case of $w = 2$ and $e = 2$, $\hat{g} = O(\log n)$.*

The issue of duplicates. One subtle issue that gets overlooked by the above analysis is the occurrence of duplicate rows in H . Indeed, in the context of the induced graph G_H , this would amount to a cycle of length 2, something that would not be considered. However, this event turns out to be the most likely failure scenario in our case and one we analyze separately.

LEMMA 3.4. *The probability that there exists a pair of rows in the sparse part of H that are identical is upper-bounded by $O(n^{2-w})$ for $w \ll n$.*

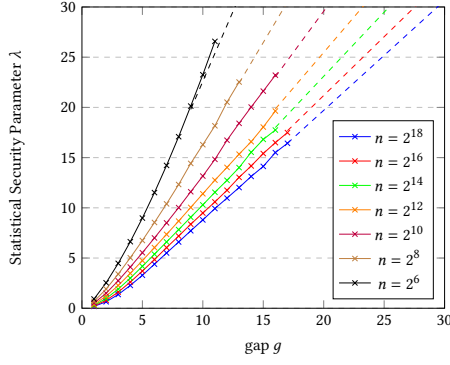


Figure 2: Security parameter λ for $w = 2$ and various g, n .

PROOF. The probability that there exists a pair of rows in the sparse part of H that are identical for $w \ll n$ is upper-bounded by

$$\binom{n}{2} \frac{\binom{m'}{w} \left[\binom{m'}{w} \right]^{n-2}}{\left[\binom{m'}{w} \right]^n} = \frac{\binom{n}{2}}{\binom{m'}{w}} \approx \frac{n^2}{2} \cdot \frac{w!}{m'^w} = O\left(\frac{1}{n^{w-2}}\right)$$

For $w = 2$, the above probability is a constant. Thus, in expectation, we will see a constant number of duplicate rows in H , a case our algorithm will not be able to handle. Indeed, as we increase w , the probability of such duplicates occurring falls significantly. This is yet another reason to consider higher weight $w > 2$.

Concrete Parameters. While the analysis above provides asymptotic guarantees that $\hat{g} = O(\log n)$ for $e = 2$, the concrete security remains ambiguous. We address this by performing extensive empirical evaluations to determine the constants involved. The general strategy is to run the triangulation algorithm a large number of times on random instances for various n . A gap of g is then given an estimated statistical security of λ bits if $2^{-\lambda}$ fraction of the instances had a gap of at most g . Figure 2 shows the resulting distribution. To ensure statistical accuracy, instances with fewer than 2^4 instances are not plotted, e.g. for $n = 2^{10}$ approximately 2^{27} trials were performed while only $\lambda \leq 23$ is plotted.

We observe two primary relationships:

- (1) For any fixed n there is generally a linear relationship between λ and g with the exception near zero which is not a practical concern. Since $\lambda = O(\log n)$ this is consistent with the theoretical analysis.
For each n we interpolated the best fit line for $g > 5$. Moreover, we observe that all of these linear relationships pass near the point $f = (1.9, 0)$. We conjecture that this is a general trend and plot the best fit line which passes through f as dashed lines.
- (2) Observe that the slope of the line is λ/g and that we expect this to be linear in $1/\log n$. Indeed, Figure 8 depicts the slopes of the best fit lines of Figure 2 and which follows this relationship. We interpolate this relationship as

$$\lambda/g = a/(\log_2 n - c) + b \quad (1)$$

where $a := 7.529, b = 0.610, c = 2.556$ and is depicted as the dashed line.

In summary, we propose a linear lower bound $\lambda \geq \alpha_n g - \alpha_n 1.9$ where $\alpha_n := a/(\log_2 n - c) + b$.

3.2 General Weight

We now consider the case of row weight $w > 2$. As before, in order to analyze the gap g , we reformulate our problem as a graph problem as follows. We sample the sparse part of H as follows: Each row is sampled independently at random subject to the constraint that the row is of weight $w > 2$. Now, consider the hypergraph $G_H = (V, E)$ induced by the sparse part of H where $V = [m']$ and

$$E = \{e_i = (e_{i,1}, \dots, e_{i,w}) : H_{i,1} = \dots = H_{i,w} = 1\}_{i \in [n]}$$

Once again, the problem of determining how to best solve the system of equations determined by H using the process of back-substitution, that is, finding free variables in H , is equivalent to the problem of optimally *peeling* G_H , where peeling G_H corresponds to iteratively selecting a hyperedge with a vertex of degree 1 (or a *free vertex*) and removing that hyperedge. If this process terminates with all the hyperedges of the hypergraph being removed, the hypergraph is said to have been completely peeled. Clearly, in our setting, this means that there is a way to solve the system of linear equations involving H entirely using back-substitution, or in other words, $g = 0$. Otherwise, the peeling algorithm terminates with a sub-hypergraph of G_H where every vertex has degree at least 2. That sub-graph is called the *2-core* of G_H .

Indeed, different sub-graphs of G_H would admit different peelings that would lead to different such final unpeelable sub-graphs. One can then ask the question of whether one can determine the largest sub-graph of G_H that can be completely peeled. We call this the problem of optimally peeling G_H . Optimally peeling G_H would lead to the identification of a sub-graph (where every vertex has degree at least 2) of minimum size which if removed from G_H would make it completely peelable. It is easy to see that the gap g is the size of this subset, i.e., the minimum number of hyperedges to be removed from this 2-core in order to make it peelable. Immediately, g is upper-bounded by the size of the 2-core of G_H . Thus, again, one way of upper-bounding g is to get a handle on the size of the 2-core of G_H . For $w = 2$, it was possible to efficiently compute the optimal peeling of G_H using a greedy algorithm. It turns out that for the case of $w > 2$, it is unlikely that there exists a general procedure to efficiently compute the optimal peeling of G_H . We go on to describe why this is the case.

Hardness of General Weight. In Section B, we describe why the matroid-inspired greedy algorithm that worked in the case of $w = 2$ does not work for the case of $w > 2$. Given this, one could ask if there is any other efficient algorithm to optimally peel G_H for the case of $w > 2$. This problem is closely connected to the problem of finding minimum unsatisfiable cores in the context of satisfiability. Two works in this regard are those of Zhang, Li and Shen [30] and Papadimitriou and David Wolfe [22]. From the first, we note that even verifying minimum cores is very hard, and that it is known to be D^P -complete from the latter ($D^P = \{A \cap B : A \in \text{NP} \wedge B \in \text{coNP}\}$). So this problem is certainly hard and is unlikely to admit efficient algorithms.

Thus, given that determining g optimally might be difficult, and that the running time of our algorithm depends quadratically on g , we are left with two questions:

- How large is g likely to be for the matrices that we sample?
- How does this affect the performance of our algorithm?

Phase Transition. The first observation we make is regarding the phenomenon of a sharp phase transition with regards to the emptiness of the 2-core that occurs with varying the expansion factor e . This phenomenon has been completely described in the work of Dembo and Montanari [7]. We recall their result in the following lemma.

LEMMA 3.5 ([7]). *For a uniformly chosen random hypergraph of $m' = en$ vertices and n hyperedges, each consisting of the same fixed number $w \geq 3$ of vertices, the size of the 2-core exhibits for large n a first-order phase transition, changing from $o(n)$ for $e > e_c$ to a positive fraction of n for $e < e_c$, with a transition window size $\Theta(n^{-1/2})$ around $e_c > 0$. For instance, for $w = 3$, $e_c \approx 1.2218$.*

Structures in the Gap. We now know that below a certain critical expansion threshold, the 2-core of G_H becomes non-empty. What does this 2-core of G_H contain? For general w , the minimal larger structures “induce” cycles in the hypergraph $G_H = (V, E)$ induced by the sparse part of H where $V = [m']$ and

$$E = \{e_i = (e_{i,1}, \dots, e_{i,w}) : H_{i,1} = \dots = H_{i,w} = 1\}_{i \in [n]}$$

Induced cycles of size 2 correspond to duplicates, which have been analyzed above (and will be discussed ahead).

LEMMA 3.6. *For $k > 2$, the probability that G_H contains an induced cycle of size k is upper-bounded by $O(\tilde{k}^{-k-\frac{1}{2}})$, where $\tilde{k} = O_{e,w}(k)$.*

PROOF. For $k > 2$, the probability that G_H contains an induced cycle of size k is upper-bounded by

$$\begin{aligned} \binom{n}{k} \frac{\binom{m'}{k} k! \left[\binom{m'-2}{w-2} \right]^k \left[\binom{m'}{w} \right]^{n-k}}{\left[\binom{m'}{w} \right]^n} &= \frac{\binom{n}{k} \frac{m'!}{(m'-k)!} \left[\binom{m'-2}{w-2} \right]^k}{\left[\binom{m'}{w} \right]^k} \\ &< \frac{\binom{n}{k} \frac{m'!}{(m'-k)!} \left[\binom{m'}{w-2} \right]^k}{\left[\binom{m'}{w} \right]^k} \end{aligned}$$

This probability is upper-bounded by

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\binom{n}{k} \frac{m'!}{(m'-k)!} \left[\binom{m'}{w-2} \right]^k}{\left[\binom{m'}{w} \right]^k} &= \frac{\left(\frac{w(w-1)}{e} \right)^k}{k!} \\ &\approx \frac{1}{\sqrt{2\pi k}} \left(\frac{\exp(1) w(w-1)}{e k} \right)^k = O\left(\frac{1}{\tilde{k}^{k+\frac{1}{2}}} \right) \end{aligned}$$

where $\tilde{k} = \beta(w, e)k$ for some $\beta(w, e) = \Theta(e w^{-2})$.

The above analysis qualitatively states that the minimal structures in the gap are more likely to be small than large. The understanding of this fact will be important as we move ahead.

The issue of duplicates. As before, the issue that gets overlooked by the above analysis is the occurrence of duplicate rows in H . From before, The probability that there exists a pair of rows in the sparse part of H that are identical is $O(n^{2-w})$ for $w \ll n$.

Putting it all together. Let us now try to describe the what we expect should happen to g as we vary e .

- Firstly, there is a phase transition window to the far right of which the only bad structures in the 2-core would be duplicates (with a certain probability) and to the far left of which the 2-core would start to contain structures of linear size.
- Secondly, the width of this phase transition varies as $\Theta(n^{-1/2})$, being wider for smaller n and narrower for larger n . Just to the right of the phase transition window, the 2-core contains, with overwhelming probability, only a few small structures.
- Through the phase transition, we would expect to see larger structures in the gap as we get closer to the left end of the phase transition window. In other words, towards the right end of the phase transition window, we would still expect that the gap is small with overwhelming probability.

Concrete Parameters. The empirical measurements we make do indeed match with the theoretical analysis above. Figure 3 plots the log failure probability versus the expansion factor $e = m'/n$ for $w = 3$ and various n . For each n we plot several curves. The lower solid curve for each n plots the security parameter for $g < 1$. Some n have a second solid curve which plots the security parameter for $g < 2$. Inspecting the distribution of these curves, we see several trends. The first is that as n increases, the curve increases at a higher rate (in other words, the phase transition window is wider for smaller n).

Secondly, there is a plateauing effect where a given curve starts to flatten out. For example, consider $n = 2^{10}$ and observe that the $g < 1$ curve flattens out at $\lambda \approx 10$ and $e = 1.31$. However, the $g < 2$ curve continues to increase. This plateauing effect perfectly agrees with the probability of small structures. The most likely (virtually only) structures in the gap post the phase transition are duplicate rows which occur with probability approximately $1/n^{w-2} = 1/n$ for $w = 3$. Therefore it is expected to have plateaus at $\lambda = n$ for each $g < 1$ curve. Similarly, the mostly likely structure of size $g < 2$ are two duplicates which, as expected, occurs at $\lambda = 2n$. Figure 3 depicts the projected continuations of these plateaus as the horizontal dashed lines.

The next observation is that, if we ignore the plateau effect, the curves for each n and $g < 1, 2, \dots$ converge to a linear growth rate. We project this growth rate for each n as the corresponding linear dashed line. The trend that each of these lines pass through the point $f = (1.223, -9.2)$, which we call the *phase transition point*. Therefore as n goes to infinity, we would expect a sharp phase transition where g goes from $g > 0$ to $g = 0$ at $e = 1.223$. This closely agrees with [7] which analytically computes the phase transition to occur at $e = 1.2218$.

Figure 10 contains a similar plot for $w = 5$. We observe that it generally follows the same trend. The most significant change is that the phase transition point f is shifted right to $f = (1.427, -9.2)$. Secondly, the plateau effect occurs at a higher λ . For example, a duplicate occurs at $\lambda \approx 3 \log(n)$ as the probability of duplicate rows for $w = 5$ is $\approx 1/n^{w-2} = 1/n^3$, which can be observed in Figure 10 for $n = 2^6$. Additionally, we performed this analysis for $w \in \{3, 4, 5, 7, 9, 14, 20, 100\}$ and observe that slope of the linear trend lines changes as w is changed.

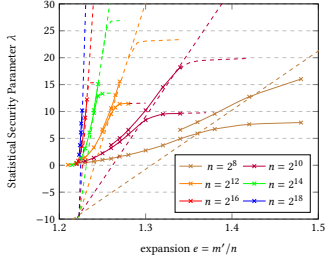


Figure 3: Security parameter λ for $w = 3$ and various e, n .

Our goal is then to translate these observations into a function of w, e, n which closely lower bounds λ in the range $30 \leq \lambda \leq 128, 3 \leq w \leq 10, 2^6 \leq n \leq 2^{30}$. The general strategy is to upper bound the e coordinate of the phase transition point $f = (e^*, -9.2)$ and lower bound the slope of the linear trend lines. These bounds can then be combined with the well defined impact of small structures and plateaus to obtain a lower bound of λ .

For all $w \in \{3, 4, 5, 7, 9, 14, 20, 100\}$ we observe that the e coordinate of the phase transition point $f = (e^*, -9.2)$ has a positive correlation with w . Figure 9 depicts this relation by plotting e for various w . As can be seen, the relationship is roughly linear. Recall that our objective is derive a function which will lower bound λ given the other parameters, and therefore we need to upper bound e in the curve in Figure 9. We achieve this in a piecewise manner with cases $w = 3, w = 4, w \geq 5$. For the first two cases we simply take the empirically obtained values while for $w \geq 5$ we propose the upper bound

$$e^* = \begin{cases} 1.223 & \text{if } w = 3 \\ 1.293 & \text{if } w = 4 \\ 0.1485w + 0.6845 & \text{otherwise} \end{cases} \quad (2)$$

This upper bound is depicted as the dashed line in Figure 9. As can be seen at $w = 20$, the empirically obtained values have a slightly sublinear growth which was further validated at $w = 100$. Since we are primarily interested in $w \leq 10$, we argue that this upper bound is sufficiently accurate.

Finally, we turn our attention to understanding the distribution of the slope (λ/e) of the linear trend lines. Figure 11 shows this relationship between the log slope and $\log n$ for various w . It is immediately clear that there is a linear relationship between $\log(\lambda/e)$ and $\log(n)$. The slope of this relationship is 0.55. This closely agrees with the predicted value of $1/2$ from [7]. Indeed, [7] states that the width of the phase transition varies as $\Theta(n^{-1/2})$. The slope of the phase transition line is inversely related to the width of the phase transition window, that is, $\lambda/e \propto \Theta(n^{1/2})$. Thus, theoretically, the slope of the linear relationship between $\log(\lambda/e)$ and $\log(n)$ would be $1/2$, which closely matches our empirical value of 0.55.

As expected, the slope is the same for all measured w but each line has a different additive offset. To understand this offset we plot the same measurements in Figure 12 where we now compare the log slope with $\log w$ for various n . The linear growth rate in Figure 11 corresponds to even spacing between all the curves. We observe that $w \in \{3, 4, 5\}$ all have approximately the same slope with $w = 4$ being slightly higher. Then as w increases we observe a general trend of the slopes decreasing. Again, our goal is to lower

bound the slope. We achieve this via the polynomial

$$\log_2(\lambda/e) = 0.55 \log_2(n) + 0.093w'^3 - 1.01w'^2 + 2.92w' - 0.13 \quad (3)$$

for the range $3 \leq w \leq 20$ and $w' := \log_2(w)$. We note that outside this range the polynomial quickly becomes an upper bound and further investigation is required to obtain a more general lower bound. We plot this lower bound in Figure 12 as the dashed lines. In summary, one can solve Equation 2 and 3 to obtain the phase transition line, then compute e and $\hat{g} = \lfloor \frac{\lambda}{(w-2) \log_2(en)} \rfloor$.

3.3 Performance of our algorithm

Optimal for $g = 0$. Our first positive result for general w is that our algorithm is optimal for the case of $g = 0$. Revisiting step 2 of the algorithm, note that for $g = 0, \delta = n$. This means that the rows can be ordered such that every row of H has a corresponding sparse column which contains a non-zero entry in that row and some of the subsequent rows. This corresponds exactly with the case of the hypergraph induced by the sparse part of H having an empty 2-core and hence can be completely peeled. Indeed, in this case, the optimal way of solving the system is via back-propagation which is done in $O(m)$ time by step 5 of our algorithm.

We would like to argue that if H had a triangulation that enabled $g = 0$, step 2 of our algorithm indeed finds one such triangulation.

LEMMA 3.7. *If H sampled in Step 1 of the procedure in Figure 1 has a triangulation with $g = 0$, Step 2 finds one such triangulation.*

PROOF. This is in fact easy to see. If H admitted such a triangulation, then there must exist a column of H that has a single non-zero entry. Thus, the first iteration of our algorithm would pick out such a column. Let us now complete the argument inductively. Consider H' after one row of H has been removed, that is, a row with a column whose only non-zero entry lies in that row. Note that H is triangularizable with $g = 0$ iff H' is. Thus, to complete the inductive argument, our algorithm optimally triangulates H' with $g = 0$, and then simply re-introducing the removed row preserves the triangulation as the row has a corresponding column whose only non-zero entry lies in that row. Alternatively, we can view this via the lens of optimally peeling G_H , i.e., step 2 of our algorithm completely peels G_H if it can be peeled.

The case of non-duplicates. If the structures in the 2-core include more than just duplicate rows, it is hard to theoretically bound the quality of the g determined by our algorithm. Indeed, our algorithm does perform better than prior works as the upper bound $\hat{g}$ on the gap g as determined by our algorithm is strictly smaller than the size of the 2-core of G_H . However, in the worst case, $\hat{g}$ could end up being as large as the size of the 2-core (minus one). While this does seem less than optimal, it is unclear if one can do significantly better (efficiently in terms of g) than this on account of the hardness results presented in Section 3.2. We discuss this in more detail ahead.

Probabilistically good approximation. The silver lining is that probabilistically, our algorithm is guaranteed to perform really well. Our analysis from before has left us with the understanding that the probability that the 2-core of G_H contains large structures declines (slightly faster than) exponentially with their size. Therefore, we are guaranteed that with high probability, the structures in the

2-core are indeed small and few in number. Furthermore, as empirically verified above, we observe that duplicates and small cycles (contributing a gap of 1 or 2) are the most likely structures in the 2-core of G_H and our algorithm works optimally in the case of just duplicates. Although the asymptotic analysis is not tight, it does provide enough theoretical evidence that g will be small (e.g. < 12), and empirically, we note that g is even smaller (e.g. 2 or 3).

Finally, the above analysis also heuristically guarantees that our algorithm, with overwhelming probability, will not veer off significantly from the actual gap. The reason for this is the following. Assume that our algorithm has to make a choice at some stage in step 2 on which column to pick among many of equal minimum non-zero weight. In the case that it makes a “wrong” choice that results in a large gap, this would mean that the sub-matrix H' (where remaining rows follow the same distribution as only the column weight has been affected by removing rows in step 2) would have to contain a sufficient number of large-enough independent structures, which, again, as argued above, is unlikely. This provides theoretical evidence for our empirical finding that our algorithm indeed performs near-optimally.

Optimality for small g . We finally discuss the optimal triangulation described in Figure 5.

LEMMA 3.8. *The procedure in Figure 5 optimally triangulates H .*

PROOF. Optimally peeling G_H amounts to identifying a minimum subset of hyperedges of the 2-core of G_H which when removed make G_H completely peelable. Indeed, if we had an upper bound $\hat{g}$ on the size of that subset of the 2-core, an exhaustive search among the edges of the 2-core would reveal the optimum (minimum) subset of hyperedges (which directly maps to the gap of H). Given the hardness results presented in Section 3.2, an exhaustive search is probably the best one could hope for, and this is exactly what is done in Figure 5. The first steps end with H' containing those rows corresponding to the 2-core of G_H . Subsequently we perform an exhaustive search to determine the optimum subset of rows.

Let h_2 denote the size of the 2-core of G_H . Then, the complexity of Figure 5 would be $O(m + h_2^{\hat{g}})$. From all of the analysis above, indeed in the parameter regime we consider, $\hat{g} = O(1)$ and $h_2 \leq e \cdot \hat{g}$ with overwhelming probability for $e = O(1)$.

4 PRIVATE SET INTERSECTION

Next we turn to the main application for our construction, efficient Private Set Intersection. PSI allows two parties to compute the intersection of their respective sets X, Y without revealing anything but the final intersection. The full functionality is detailed in Figure 6. To realize this functionality we make use of the recent PSI construction of [28] and modify it in two ways.

- (1) Replace the OKVS of [24] with our new construction.
- (2) Extend the PSI protocol of [28] to use subfield-VOLE [5].

As detailed below, the former alteration improves both the computational overhead and communication overhead. In particular, when we instantiate the [28] PSI protocol with our new OKVS, the PSI protocol is able to be executed on a consumer laptop in 0.4

seconds on a single thread, or 0.16 seconds with 4 threads. Additionally, when the subfield-VOLE optimization is applied we are able to further reduce the communication overhead.

Our starting point is the [subfield] VOLE protocol of [2, 5]. The VOLE functionality described in Figure 7 consists of a Sender and Receiver. The Sender is output a random $\Delta \in \mathbb{F}$, $\vec{B} \in \mathbb{F}^m$ while the Receiver is output random vectors $\vec{A} \in \mathbb{B}^m$, $\vec{C} \in \mathbb{F}^m$ subject to

$$\vec{C} - \vec{B} = \Delta \vec{A}$$

where $\mathbb{F}$ is some extension field of $\mathbb{B}$. For example, $|\mathbb{B}| \approx 2^{\kappa/2}$ while $\mathbb{F}$ is the degree 2 extension. To obtain PSI, the intuitive idea is to derandomize the VOLE correlation $(\vec{A}, \vec{B}, \vec{C}, \delta)$ to $(\vec{P}, \vec{B}', \vec{C}')$ where $\vec{C}' - \vec{B}' = \Delta \vec{P}$ and $\vec{P}$ is a OKVS which decodes to $H^{\mathbb{B}}(x)$ for all $x \in X$, where $H^{\mathbb{B}} : \{0, 1\}^* \rightarrow \mathbb{B}$ is a random oracle. This derandomization is the main communication overhead and requires sending $\vec{A} - \vec{P}$, which requires sending $m \log_2 |\mathbb{B}|$ -bits.

The next phase takes advantage of OKVS linearity. In particular, for $x \in X$ we have the following qualities

$$\vec{C}' - \vec{B}' = \Delta \vec{P}$$

$$\text{Decode}(\vec{C}', x) - \text{Decode}(\vec{B}', x) = \Delta \text{Decode}(\vec{P}, x) = \Delta H^{\mathbb{B}}(x)$$

$$\text{Decode}(\vec{C}', x) = \text{Decode}(\vec{B}', x) + \Delta H^{\mathbb{B}}(x)$$

The Sender with $\vec{B}'$, Δ is able to compute the right hand side while the Receiver with $\vec{C}'$ is able to compute the left hand side. The Receiver computes $X' := \{F(x) \mid x \in X\}$ where $F(x) := H^0(\text{Decode}(\vec{C}', x))$, while the Sender can compute $F(y) = H^0(\text{Decode}(\vec{B}', y) + \Delta H^{\mathbb{B}}(y))$ for any y . Here, $H^0 : \{0, 1\}^* \rightarrow \{0, 1\}^{\text{out}}$ is another random oracle where $\text{out} = \lambda + \log_2(n_x) + \log_2(n_y)$. The PSI protocol completes by having the Sender send $Y' := \{F(y) \mid y \in Y\}$ to the Receiver who can then infer the intersection $X \cap Y$ from $X' \cap Y'$. The full semi-honest protocol is presented in Figure 4. Next we describe several ways our protocol can be instantiated (by defining $\mathbb{B}$, $H^{\mathbb{B}}$ and the OKVS parameters) to get different tradeoffs.

Our fast instantiation. The more efficient instantiation of our protocol in terms of running time is to define $\mathbb{B} = GF(\kappa)$, $\mathbb{F} = \mathbb{B}$. This variant does not use subfield VOLE due to it imposing is slight running time overhead. We also optimize our OKVS scheme to minimize running time at a cost of mildly increasing m . In particular we employ our $w = 3$ scheme with a cluster size of 2^{14} . This results in $m \approx 1.28n$. See Theorem C.1 for the proof of security.

Low communication instantiation. Next we aim to minimize the communication overhead at the mild expense of running time. First we will instantiate $\mathbb{B}$ as the smallest field such that $|\mathbb{B}| \geq 2^{\lambda + \log_2(n_x n_y)}$ and $\mathbb{F}$ as the smallest extension such that $|\mathbb{F}| \geq 2^{\kappa}$. The size requirement on $\mathbb{B}$ stems from need to ensure that random collisions do not occur which would result in leaking some information about Y or the receiver outputting an x which is not in Y . The intuition is that there are $n_x n_y$ possible collisions of this type which necessitates $\mathbb{B}$ elements to be statistical security parameter plus $\log_2(n_x n_y)$ bits in length. Additionally, we parameterize the OKVS scheme to minimize the size. This effectively means to not use clustering and results in $m = 1.23n$.

See Theorem C.1 in Section C.1 for the proof of security.

Parameters: There are two parties, a Sender and a Receiver with a respective set Y, X where $|Y| = n_y$, $|X| = n_x$. Let $\mathbb{B}$ be a field with extension $\mathbb{F}$ such that $|\mathbb{F}| = O(2^\kappa)$. Let $H^{\mathbb{B}} : \{0, 1\}^* \rightarrow \mathbb{B}$, $H^{\mathbb{O}} : \{0, 1\}^* \rightarrow \{0, 1\}^{\text{out}}$ be random oracles.

Protocol: Upon input (sender, sid, Y) from the Sender and (receiver, sid, X) from the Receiver:

- (1) The Receiver samples $r, r' \leftarrow \{0, 1\}^\kappa$ and computes $\vec{P} := \text{Encode}(L, r)$ where $L := \{(x, H^{\mathbb{B}}(x, r')) \mid x \in X\}$.
- (2) The Sender sends (sender, sid) and the Receiver sends (receiver, sid) to $\mathcal{F}_{\text{sub-voles}}$ with dimension $m := |\vec{P}|$, base field $\mathbb{B}$ and extension $\mathbb{F}$. The parties respectively receive $\Delta, \vec{B}$ and $\vec{C} := \vec{A}\Delta + \vec{B}, \vec{A}$.
- (3) The Receiver sends $r, \vec{A}' := \vec{A} - \vec{P} \in \mathbb{B}^m$ to the Sender who defines $\vec{B}' := \vec{B} + \vec{A}'\Delta \in \mathbb{F}^m$.
- (4) The Sender sends $Y' := (H^{\mathbb{O}}(\text{Decode}(\vec{B}', y, r) - \Delta H^{\mathbb{B}}(y, r')) \mid y \in Y)$ to the Receiver in a random order.
- (5) The Receiver outputs $\{x \in X \mid H^{\mathbb{O}}(\text{Decode}(\vec{C}, x, r)) \in Y'\}$.

Figure 4: Semi-honest protocol Π_{psi} .

Malicious instantiation. For malicious security we directly use the protocol of [28] with our OKVS scheme.

Circuit PSI. We make use of the Circuit PSI protocol of [28]. We note that this protocol requires an additional *oblivious* property from the OKVS. See Section A.2 or [28] for details.

5 EVALUATION

We now turn our attention to the concrete performance of our constructions and a comparison to related work. In summary, our constructions outperform all prior works by a significant margin. We used the existing implementations from [4, 27]. The protocols were all bench-marked on a single laptop with Intel i7 9750H and 16GB of RAM. All constructions target $\lambda = 40$ bits of statistical security and $\kappa = 128$ bits of computational security. All protocols were run in the $\sim 1\text{Gbps}$ LAN setting with sub millisecond latency. MT denotes that 4 threads per party are used.

5.1 OKVS

We begin by comparing the running time of the encoding and decoding of our constructions and that of [12]. As shown in Figure 1, our $w = 3$ constitution requires 653 milliseconds to encode 2^{20} items and just 54 milliseconds to decode them. This is $13\times$ and $64\times$ faster than the ($w = 3$) construction of [12], respectively. We attribute this speedup to several factors: smaller expansion, approximately $20\times$ fewer dense columns and our faster encoding algorithm. Our construction which uses clustering with size $\beta = 2^{14}$ requires just 143 milliseconds compared to 4,912 for the star construction of [12], a speedup of $34\times$. We attribute this additional speedup, as compared to $w = 3$, to their star construction requiring a two phase encoding in order to amplify the success probability to be at least $1 - 2^{-\lambda}$. Moreover, their construction & parameters are not optimized for memory efficiency. In addition, our basic $w = 3$ construction is fully secure and therefore our construction allows encoding to be performed in a single pass, with independence between the clusters. This independence property enables multi-threading which can provide an additional $3\times$ speedup with 4 threads.

Table 1: The running time (ms) of encoding and decoding various constructions and input sizes n . The elements are $\mathbb{F}_{2^{128}}$ for our construction and $\mathbb{F}_{2^{64}}$ for [12]. Binary row denotes that the dense columns are binary and opposed to the default of field row, see Section 2.

Construction	Encode (ms)			Decode (ms)		
	2^{16}	2^{20}	2^{24}	2^{16}	2^{20}	2^{24}
Ours ($w = 2$)	22.4	878	14,729	7.1	165	3,390
Ours ($w = 3$)	12.1	653	15,362	2.0	54	1,394
Ours ($w = 3$, binary row)	13.4	680	15,832	3.6	82	1,922
Ours ($w = 5$)	20	1251	30,206	6.4	144	3,317
Ours ($w = 3$, cluster)	9.5	143	2,412	1.7	42	930
Ours ($w = 3$, cluster, MT)	6.3	47	764	1.1	14	379
[12] ($w = 3$)	359	8,420	158,165	195	3,484	61,014
[12] ($w = 3$, star)	419	4,912	—	282	5,475	—

We also consider the running time of our construction with $w \in \{2, 5\}$ and observe that $w = 3$ outperforms them in terms of running time and compactness. As such we conclude that $w = 3$ is the correct choice for most applications. However, we note that $w > 3$ can have advantages in that larger w can allow for the complete removal of all dense columns. Therefore, if the overall weight needs to be minimized, then $w = 5$ or larger may be preferable.

5.2 Private Set Intersection

As discussed in Section 4, we proposed several instantiations of our PSI protocols. The first is our “fast” variant which is optimized for small running time. The second version, dubbed “SD”, uses the small domain optimization which achieves low communication overhead. “MT” denotes that 4 threads per party were used.

In all cases our protocols significantly outperform the competition. The previous fastest semi-honest protocol was that of [16] which uses a specialized OPRF-type protocol. Our protocols are between 3 to $8\times$ faster on a single thread. Our protocol sends approximately 2 to 5 times less data. Moreover, our fast protocol (with clustering) enables a multi threaded implementation, which brings the overall running time to 0.16 seconds for $n = 2^{20}$, a $13\times$ speedup. Compared to [28] on which our protocol is based, at $n = 2^{20}$ we observe a speedup of $125\times$ for our protocol and $40\times$ for our low communication protocol while reducing the communication overhead $1.7\times$ and $2.2\times$, respectively.

Our PSI protocols also outperform [12] in running time and communication by several fold. [12] is based on the PSI protocol of [24] with the original $w = 2$ OKVS scheme being replaced with $w = 3$, star scheme discussed above. Finally, in the malicious setting we observe that our protocol has almost no additional overhead. As such, we similarly outperform all prior works by several fold in running time and communication.

Our protocol can also be faster than so called Naive-PSI, where the sender sends the hash of their items. We are faster this insecure protocol when SHA256 is used. Compared to a slightly less naive hashing (AES-hashing), the running time and communication could be improved to be about $2\times$ better than ours. For unbalanced set sizes ($|X| \gg |Y|$), naive hashing is still a big improvement due to communication scaling with $|Y|$ and not $|X| + |Y|$.

Finally, it can be seen that in Table 2 that our circuit PSI protocol significantly outperforms the prior art in terms of running time and communications. Depending on what we compare with, our protocol is between 1.5 and 5 times faster and sends 15 to 2 times

Table 2: Performance metrics of our PSI protocols compared to related works.

Protocol	Time (ms)			Comm. (bits/ n)			Comm. asymptotic (bits) n_x, n_y
	2^{16}	2^{20}	2^{24}	2^{16}	2^{20}	2^{24}	
Semi-Honest PSI							
[16]	137	2,073	53,933	$984n$	$1008n$	$1032n$	$6\kappa n_x + 3(\lambda + \log(n_x n_y))n_y$
[24] ($w = 2$)	763	4,998	123,800	$1208n$	$1268n$	$1302n$	$9.3\kappa n_x + (\lambda + \log(n_x n_y))n_y$
[12] ($w = 3$, star)	180	2,268	—	$780n$	$788n$	$804n$	$5.6\kappa n_x + (\lambda + \log(n_x n_y))n_y$
[28] ($w = 2$)	499	4,580	113,994	$914n$	$426n$	$398n$	$2^{24} n_x^{0.05} + 307n_x + 40n_y + \log(n_x n_y)n_y$
Ours (fast)	51	369	6,987	$241n$	$251n$	$260n$	$1.3\kappa n_x + (\lambda + \log(n_x n_y))n_y + 2^{14.5}\kappa$
Ours (fast, MT)	49	163	3,145				
Ours (SD)	63	1,353	27,681	$206n$	$180n$	$196n$	
Ours (SD, MT)	61	1,107	25,325				
Malicious PSI							
[24] ($w = 2$)	769	5,196	126,294	$1766n$			$11.8\kappa n_x + 2\kappa n_y$
[12] ($w = 3$, star)	184	2,291	—	$1357n$			$8.6\kappa n_x + 2\kappa n_y$
[28] ($w = 2$)	556	5,228	132,951	$960n$	$474n$	$438n$	$2.4\kappa n_x + \kappa n_y + 2^{17}\kappa n_x^{0.05}$
Ours	62	439	8,055	$343n$	$302n$	$300n$	
Ours (MT)	65	222	3,984				
Semi-Honest Circuit PSI							
[28] ($w = 2$, IKNP)	1,810	25,300	—	$21,888n$	$14,640n$	—	$O(n\kappa\ell)$
[28] ($w = 2$, SilentOT)	5,021	112,421	—	$2,701n$	$2,216n$	—	$O(n\ell)$
[4] (IKNP+)	2,851	28,723	—	$8,371n$	$8,856n$	—	$O(n\kappa\ell)$
[4] (Silver) –Estimated–	2,337	23,840	—	$8,371n$	$8,856n$	—	$O(n\ell)$
Ours ($w = 3$, Silver)	1,112	15,557	—	$931n$	$921n$	—	$O(n\ell)$

less data. [4] is arguably the most competitive, requiring 1.8× more running time and 10× more communication. Their protocol uses IKNP to evaluate a comparison circuit at the end of their protocol along with several optimizations. One could then ask if using SilentOT+Silver could make their protocol competitive with ours. Their asymptotic communication would reduce to the same as ours, $O(n\ell)$. However, we argue that it would remain concretely higher due the added overhead of their “relaxed OPPRF” which necessitates comparing each output with three values, compared to our protocol comparing with just one. Moreover, their running time would likely remain about the same with the use of Silver.

REFERENCES

- [1] Béla Bollobás. 1984. The Evolution of Random Graphs. *Trans. Amer. Math. Soc.* 286, 1 (1984), 257–274.
- [2] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. 2019. Efficient Two-Round OT Extension and Silent Non-Interactive Secure Computation. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11–15, 2019*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM, 291–308.
- [3] Prasad Buddhavarapu, Andrew Knox, Payman Mohassel, Shubho Sengupta, Erik Taubeneck, and Vlad Vlasnik. 2020. Private Matching for Compute. *IACR Cryptol. ePrint Arch.* 2020 (2020), 599.
- [4] Nishanth Chandran, Divya Gupta, and Akash Shah. 2022. Circuit-PSI with Linear Complexity via Relaxed Batch OPPRF. *Proceedings on Privacy Enhancing Technologies (PETs)*. <https://doi.org/10.2478/popets-2022-0018> <https://eprint.iacr.org/2021/034>.
- [5] Geoffroy Couteau, Peter Rindal, and Srinivasan Raghuraman. 2021. Silver: Silent VOLE and Oblivious Transfer from Hardness of Decoding Structured LDPC Codes. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 12827)*, Tal Malkin and Chris Peikert (Eds.). Springer, 502–534.
- [6] Emiliano De Cristofaro and Gene Tsudik. 2010. Practical Private Set Intersection Protocols with Linear Complexity. In *Financial Cryptography (Lecture Notes in Computer Science, Vol. 6052)*. Springer, 143–159.
- [7] Amir Dembo and Andrea Montanari. 2008. Finite size scaling for the core of large random hypergraphs. *The Annals of Applied Probability* 18, 5 (2008).
- [8] Paul Erdős and Alfred Rényi. 1959. On random graphs I. *Publ. Math. Debrecen* 6 (1959), 290–397.
- [9] Quentin Fortier. 2017. Aspects of connectivity with matroid constraints in graphs Modeling and Simulation. *Université Grenoble Alpes NNT : 2017GREAM059* (2017).
- [10] András Frank, Tamás Király, and Matthias Kriesell. 2003. On decomposing a hypergraph into k connected sub-hypergraphs. *Discret. Appl. Math.* 131, 2 (2003), 373–383.
- [11] Alan Frieze and Michal Karoński. 2021. Introduction to Random Graphs.
- [12] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. 2021. Oblivious Key-Value Stores and Amplification for Private Set Intersection. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12826)*, Tal Malkin and Chris Peikert (Eds.). Springer, 395–425.
- [13] Yan Huang, David Evans, and Jonathan Katz. 2012. Private Set Intersection: Are Garbled Circuits Better than Custom Protocols?. In *19th Annual Network and Distributed System Security Symposium, NDSS 2012, San Diego, California, USA, February 5–8, 2012*. The Internet Society. <https://www.ndss-symposium.org/ndss2012/private-set-intersection-are-garbled-circuits-better-custom-protocols>
- [14] Mihaela Ion, Ben Kreuter, Ahmet Erhan Nergiz, Sarvar Patel, Shobhit Saxena, Karn Seth, Mariana Raykova, David Shanahan, and Moti Yung. 2020. On Deploying Secure Computing: Private Intersection-Sum-with-Cardinality. In *EuroS&P*. IEEE, 370–389.
- [15] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. 2003. Extending Oblivious Transfers Efficiently. In *CRYPTO (Lecture Notes in Computer Science, Vol. 2729)*. Springer, 145–161.
- [16] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. 2016. Efficient Batched Oblivious PRF with Applications to Private Set Intersection. *IACR Cryptol. ePrint Arch.* (2016), 799. <http://eprint.iacr.org/2016/799>
- [17] Tomasz Łuczak. 1990. Component Behavior Near the Critical Point of the Random Graph Process. *Random Struct. Algorithms* 1, 3 (1990), 287–310.
- [18] Catherine A. Meadows. 1986. A More Efficient Cryptographic Matchmaking Protocol for Use in the Absence of a Continuously Available Third Party. In *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 134–137.
- [19] Ofri Nevo, Ni Trieu, and Avishay Yanai. 2021. Simple, Fast Malicious Multiparty Private Set Intersection. <https://ia.cr/2021/1221>.
- [20] Michele Orrù, Emmanuela Orsini, and Peter Scholl. 2017. Actively Secure 1-out-of- N OT Extension with Application to Private Set Intersection. In *CT-RSA (Lecture Notes in Computer Science, Vol. 10159)*. Springer, 381–396.
- [21] J.G. Oxley. 2006. *Matroid Theory*. Oxford University Press. <https://books.google.com/books?id=puKta1Hdz-8C>
- [22] Christos H. Papadimitriou and David Wolfe. 1988. The Complexity of Facets Resolved. *J. Comput. Syst. Sci.* 37, 1 (1988), 2–13.
- [23] Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. 2019. SpOT-Light: Lightweight Private Set Intersection from Sparse OT Extension. In *CRYPTO (3) (Lecture Notes in Computer Science, Vol. 11694)*. Springer, 401–431.
- [24] Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. 2020. PSI from PaXoS: Fast, Malicious Private Set Intersection. In *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12106)*, Anne Canteaut and Yuval Ishai (Eds.). Springer, 739–767.
- [25] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. 2015. Phasing: Private Set Intersection Using Permutation-based Hashing. In *USENIX Security Symposium*. USENIX Association, 515–530.
- [26] Benny Pinkas, Thomas Schneider, and Michael Zohner. 2014. Faster Private Set Intersection Based on OT Extension. In *USENIX Security Symposium*. USENIX Association, 797–812.

- [27] Peter Rindal and Mike Rosulek. 2017. Malicious-Secure Private Set Intersection via Dual Execution. In *ACM Conference on Computer and Communications Security*. ACM, 1229–1242.
- [28] Peter Rindal and Philipp Schoppmann. 2021. VOLE-PSI: Fast OPRF and Circuit-PSI from Vector-OLE. In *Advances in Cryptology - EUROCRYPT 2021 - 40th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12697)*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer, 901–930.
- [29] Stefan Walzer. 2021. Peeling Close to the Orientability Threshold - Spatial Coupling in Hashing-Based Data Structures. In *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021*, Daniel Marx (Ed.). SIAM, 2194–2211.
- [30] Jianmin Zhang, Sikun Li, and ShengYu Shen. 2006. Extracting Minimum Unsatisfiable Cores with a Greedy Genetic Algorithm. In *AI 2006: Advances in Artificial Intelligence, 19th Australian Joint Conference on Artificial Intelligence, Hobart, Australia, December 4-8, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4304)*, Abdul Sattar and Byeong-Ho Kang (Eds.). Springer, 847–856.